

文章笔记

用 Karpathy 的 AutoResearch 方法将 Skills 效果提升 10 倍

五道口纳什 & Codex

2026 年 3 月 29 日

原文作者: Ole Lehmann (@itssolelehmann)

原文标题: How to 10x your Claude Skills (using Karpathy's autoresearch method)

译者: 实践哥 MinLi (@MinLiBuilds)

译文来源: X / Twitter

发布日期: 2026 年 3 月 19 日

译者按

本文作者 Ole Lehmann 是 AI 领域极具影响力的内容创作者，专注于 AI 商业自动化和个人生产力。文章的核心价值在于：任何可度量、可量化的 skill / 工作流 / 研究事项，都可以通过 autoresearch 方法持续优化，效果提升 10 倍。原理极其简单——评估、优化、评估、优化，无限循环。

目录

1 问题：你的 Skill 可能正在悄悄失效	2
1.1 三种隐蔽的失效模式	2
1.2 本章小结	2
2 AutoResearch 方法的起源	2
2.1 本章小结	3
3 一次循环的工作原理	3
3.1 本章小结	4
4 评分标准：唯一需要人类参与的环节	4
4.1 为什么用是/否而不用打分	4
4.2 落地页文案 Checklist 示例	4
4.3 本章小结	4
5 实操步骤	5
5.1 本章小结	5
6 实战案例：落地页文案 Skill 优化	5
6.1 Agent 做了哪些改动	5
6.2 最终产出物	6
6.3 本章小结	6
7 更广泛的应用场景	6
7.1 本章小结	7
8 总结与延伸	7
8.1 核心要点回顾	7
8.2 方法论启示	7
8.3 延伸阅读	7

1 问题：你的 Skill 可能正在悄悄失效

你的 Claude skill，有 30% 的时间都在悄悄骗人——你的客户可能比你先发现。

作者讲了一个亲身经历：他把自己搭建的落地页文案 skill 交给客户使用，信心十足地说“绝对靠谱”。结果当天晚上，客户发来截图——CTA 写的是“了解更多”，标题是“转变你的业务”。这些全是模糊、无效的模板化输出。

那一刻他意识到：**他根本不知道这个 skill 什么时候好用、什么时候在糊弄人。**

核心痛点

大多数人无法区分 AI workflow 是“表现好”还是“只是在输出文字”。大家都在教你怎么搭 AI 工具，**没人教你怎么测。**

作者随后搭了一套方法，能在 autopilot 模式下自动迭代任何 skill。他的落地页文案 skill 质量检查通过率从 **56% 涨到了 92%**，全程零手动。

1.1 三种隐藏的失效模式

作者总结了三种常见但难以察觉的 skill 失效方式：

1. **渐进漂移**：prompt 没明确禁止的内容，模型会慢慢漂向“安全感”——输出越来越模糊、越来越像模板。每次都“过得去”，但每次都差一点。等你发现时，根本不知道从第几轮开始出问题的。
2. **幸存者偏差**：你只看到“还不错”的输出——打开、用掉、关掉。那些悄悄失效的（格式跑偏、关键要素遗漏），你永远不知道它们有多频繁，因为你根本不会去翻。
3. **伪修复**：偶尔发现问题，手动改一改那次输出，告诉自己“修了”。但你改的是那一次，不是 skill 本身。下次还会在同一个地方出错。

自欺欺人的陷阱

手动修改单次输出 $\neq$ 修复 skill 本身。你改的是症状，不是根因。下一次运行，同样的问题还会出现。

1.2 本章小结

Skill 的失效往往是隐蔽的、渐进的。三种典型模式——渐进漂移、幸存者偏差、伪修复——使得用户对 skill 的实际质量产生错误的信心。核心问题在于：缺乏系统化的评估手段。

2 AutoResearch 方法的起源

Andrej Karpathy（OpenAI 联合创始人、特斯拉前 AI 负责人、“vibe coding”一词的发明者）发布了一套叫 **autoresearch** 的方法。

AutoResearch 核心思路

不让你手动改进，而是让 AI agent 在循环里替你干：

- 尝试一个小改动
- 看结果变好了没有
- 变好就留下，没变好就扔掉
- 然后再来一次，再来一次……

Karpathy 最初把这套方法用在机器学习代码上，但它适用于**任何能衡量、能改进的东西**。

背景：为什么需要自动化评估

你上了多少 AI 课、学了多少 prompt 技巧——但有人告诉过你，怎么验证你搭好的东西真的在工作吗？没有。大家都在教你怎么搭，没人教你怎么测。AutoResearch 正是填补这一空白的方法论。

作者将 Karpathy 的方法改造成了一个可以在 Claude Code 和 Cowork 里运行的 skill。使用方式极其简单：说一句”对我的落地页 skill 跑 autoresearch”，剩下的它全搞定。

2.1 本章小结

AutoResearch 源自 Karpathy，核心是”小改动 → 评估 → 保留或撤销”的自动循环。它从机器学习场景推广到任何可量化的优化任务，填补了”只教搭建、不教测试”的行业空白。

3 一次循环的工作原理

作者用”菜谱”做了一个形象的类比来解释 autoresearch 的单次循环：

菜谱类比

你有一个菜谱，十次里有七次做得不错，三次不对劲。你不是从头重写菜谱，而是：换一种配料 → 用这个改动做十次 → 变好就留下，变差就换回 → 改下一个配料 → 再做十次……经过 50 轮，你的菜谱十次里有 9.5 次都能成功。

映射到 skill 优化：

- **菜谱** = 你的 skill prompt
- **做菜** = 运行这个 skill
- **试味** = 给输出结果打分

这是一个典型的**贪心搜索**过程：每次只改一个变量，用评估结果决定保留或回退，逐步逼近最优解。

3.1 本章小结

单次循环的逻辑是：改一个点 → 测试 → 根据分数决定保留或撤销。通过大量循环的累积，skill 的整体质量从”七成靠谱”逐步提升到”九成五靠谱”。

4 评分标准：唯一需要人类参与的环节

你唯一要做的事

告诉 agent ”什么叫好”的 checklist——这是你在整个过程里唯一要干的事。用**简单的是/否问题**来定义，每个问题检查输出的一个具体方面。通过或失败，就这么简单。

4.1 为什么用是/否而不用打分

作者用”老师批卷子”来说明：不是”给写作质量打个 1-10 分”（模糊，每次结果不一样），而是每一项都清清楚楚**是或否**：

- 学生有没有写论点？是或否。
- 每处引用都注明出处了吗？是或否。
- 篇幅在 5 页以内吗？是或否。

用这份 checklist 批 100 份卷子，每次结果都一致。

4.2 落地页文案 Checklist 示例

1. ”标题有没有包含具体数字或可量化结果？”——不是”更好的文案”，而是”3 天回收广告费”。
2. ”开头第一句有没有点出一个具体的、有名字的痛苦场景？”——不是”很多人都有这个问题”，而是”你发了邮件但对方根本没回”。
3. ”CTA 是否清楚告诉用户做完这一步之后会发生什么？”——不是”立即注册”，而是”注册后 3 分钟内收到你的分析报告”。
4. ”全文有没有出现’颠覆性’行业领先’最优解’这类零信息量词汇？”
5. ”第一段内，有没有提到用户拿到结果之后的具体生活变化？”

这些 checklist 不需要你自己想。启动 autoresearch 时，agent 会全程引导你——问你什么叫好，帮你把模糊的感觉变成能打勾的问题。

Checklist 数量的陷阱

3-6 个问题是最佳数量，千万别贪多。作者试过加到 10 个，skill 开始专门”应付 checklist”，输出反而更烂——就像学生背答案、根本没理解题目。这是一种过拟合现象。

4.3 本章小结

评分标准是 autoresearch 中唯一需要人类定义的部分。使用是/否二元判断而非主观打分，确保评估的一致性和可复现性。Checklist 应控制在 3-6 个问题，避免”过拟合 checklist”的反效果。

5 实操步骤

以下是运行 autoresearch 的完整流程：

1. **下载 skill**：获取 autoresearch skill，放进 Claude Code 或 Cowork 的 skills 文件夹。
2. **选一个要改进的 skill**：说”对我的 [skill 名称] skill 跑 autoresearch”。选最让你头疼的——时好时坏、输出不稳定的那个。
3. **Agent 问你 3 件事**：要优化哪个 skill、用什么测试输入（如”为一款 AI 生产力工具写落地页文案”）、以及你的 checklist 问题是什么。
4. **跑一遍 skill，给出起始分数**：这是基准线。作者的落地页 skill 起步 56%。
5. **浏览器弹出实时 dashboard**：分数曲线、每项 checklist 的通过/失败、每次改动日志，每 10 秒刷新。
6. **走开**：agent 开始循环——找最薄弱的地方，改一点，测试，分数升就留，分数降就撤。一直跑，直到你叫停或连续三次超过 95%。

全自动运行

你可以盯着 dashboard 看，也可以去喝杯咖啡。Agent 不需要你参与任何决策——评估、改动、回退全部自动完成。

5.1 本章小结

实操分为 6 步：下载 skill → 选目标 → 回答 3 个问题 → 建立基准线 → 观察 dashboard → 放手让 agent 跑。整个过程除了初始配置外完全自动化。

6 实战案例：落地页文案 Skill 优化

作者在自己的落地页文案 skill 上运行 autoresearch 的结果：

通过率：56% → 92%

共 4 轮改动，3 个保留，1 个撤销。

6.1 Agent 做了哪些改动

1. **加明确规则**：针对最高频失败项，加了一条规则——”标题必须包含具体数字或结果。禁止使用‘转变你的业务’这类模糊承诺。”
2. **加禁用词列表**：明确列出禁用的流行词——revolutionary、cutting-edge、synergy、next-level、game-changing、leverage、unlock、transform。
3. **加高质量示例**：加了一段实际的高质量落地页示例，标注了痛点开场白和 CTA 位置，让 skill 能直接看到”好”的样子。

4. **尝试更严格字数限制（被撤销）**：文案变得太单薄，CTA 质量也下降。系统识别出了”单独看像改进、但实际损害整体输出”的改动。

改动被撤销的智慧

第 4 轮改动（更严字数限制）被自动撤销，说明系统不是盲目叠加约束，而是能识别出**局部改进但全局退步**的情况。这是 autoresearch 相比手动调优的关键优势。

6.2 最终产出物

- 改进后的 skill（原版完好，随时可恢复）
- 每一轮分数的结果日志
- 完整的 changelog：每次改动内容、原因、结果
- 原始 skill 的备份

Changelog 的价值

那个 changelog 可能是最值钱的东西——它是 skill 完整的”经验总结”，什么有用、什么没用一清二楚。等更强的模型出来，把这份 changelog 交给它，它就能从上一个 agent 停下的地方接着干。

作者说，跑完那次 autoresearch，改变最大的不是分数，是**感觉**——从”希望这次没问题”变成”我知道它在什么情况下能工作、什么情况下会出错、出错了怎么找到问题”。**从靠运气，变成靠系统。**

6.3 本章小结

实战结果证明 autoresearch 的有效性：4 轮自动迭代将通过率从 56% 提升到 92%。系统能自动识别有害改动并撤销。最终产出不仅是优化后的 skill，还包括完整的改动日志，为后续持续优化提供基础。

7 更广泛的应用场景

AutoResearch 的适用范围远不止 Claude skill。**任何能打分的东西，都能用这套方法。**

- **网站速度优化**：改一处 → 测量速度 → 留下或撤销。有人 67 轮后将页面加载时间从 1100ms 降到 67ms。
- **陌生客户开发邮件**：定义 checklist（有没有提到对方公司？是否在 75 字以内？是否以具体问题收尾？），让 agent 跑 50 个变体。
- **Newsletter 开篇**：检查”开场白有没有个人细节””有没有陈词滥调”，让 agent 在 autopilot 上打磨文字。
- **任何你反复使用的 prompt。**

通用原则

能打分，就能 autoresearch。关键前提是：你的优化目标必须是**可量化的**。

7.1 本章小结

AutoResearch 方法的适用条件是”可量化”。从网站性能到邮件写作、从 newsletter 到任何反复使用的 prompt，只要能定义出是/否评分标准，就能套用这套自动迭代框架。

8 总结与延伸

8.1 核心要点回顾

1. **问题**：大多数人不知道自己的 AI skill / workflow 是否真的有效，缺乏系统化测试手段。
2. **方法**：借鉴 Karpathy 的 autoresearch，让 AI agent 在”小改动 → 评估 → 保留或撤销”的循环中自动优化 skill。
3. **关键设计**：用 3-6 个是/否 checklist 问题定义”什么叫好”，确保评估一致、可复现。
4. **实证**：落地页文案 skill 从 56% 提升到 92%，全程零手动。
5. **普适性**：任何可量化的优化目标都适用——不限于 AI skill。

8.2 方法论启示

这篇文章最深刻的洞察不在于技术细节，而在于一个认知转变：

从靠运气到靠系统

- ”搭好就用”是第一阶段——你有了工具。
- ”搭好、测过、持续迭代”才是成熟阶段——你有了**系统**。
- Changelog 的积累让 skill 的改进成为**可传承**的资产，而非一次性的调参。

8.3 延伸阅读

- Andrej Karpathy 的 autoresearch 原始方法与仓库
- Ole Lehmann 的 X 账号：@itssolelehmann
- 译者实践哥 MinLi 的 X 账号：@MinLiBuilds